



Internship Project Report on

BookEHealth

Submitted in partial fulfilment of the requirement for the award of degree of

Master of Computer Application (MCA)

at

University of Mumbai

Submitted by

Gayatri Mahajan

Under the guidance of

Dr. Nidhi Poonia



Bharati Vidyapeeth's

Institute of Management and Information Technology

Sector 8, CBD Belapur, Navi Mumbai

1. Design Approach

Operating Tools and Development Environment

- The project uses various tools and technologies such as Java, Selenium WebDriver, TestNG, JIRA, Postman, JMeter, GitHub, .NET, Eclipse IDE, Chrome Browser, Angular, Firebase, Razorpay, and Zoom SDK for development and testing activities.
- Selenium WebDriver and TestNG are used for automation testing, JIRA is used for defect tracking, Postman is used for API testing, and JMeter is used for performance and load testing.
- GitHub is used for version control and project management, while .NET and Angular are used for application development and frontend integration.
- The development and testing environment is configured on Windows 11 using Java JDK, ChromeDriver, Selenium automation setup, Eclipse IDE, and stable internet connectivity for web and mobile application testing.

1.2. STLC (Software Testing Life Cycle) Approach

1. Requirement Analysis

- Analyze business and functional requirements of the healthcare application.
- Identify modules such as login, appointment booking, reports, chat, meetings, prescriptions, and diet plans for testing.

2. Test Planning

- Define testing scope, objectives, resources, tools, timelines, and testing strategy.
- Prepare test planning documents and identify risks and deliverables.

3. Test Case Development

- Create test cases, test scenarios, and test data for web and mobile applications.
- Prepare positive, negative, functional, UI, integration, and regression test cases.

4. Test Environment Setup

- Configure testing environment using Selenium, Java, ChromeDriver, Eclipse IDE, and testing tools.

- Verify application build, browser compatibility, internet connectivity, and test data availability.

5. Test Execution

- Execute manual and automation test cases for healthcare modules.
- Record pass/fail results, capture screenshots, and report defects in JIRA or defect tracking sheets.

6. Defect Reporting and Retesting

- Log bugs with severity and priority details.
- Retest fixed defects and perform regression testing to validate application stability.

7. Test Closure

- Prepare test summary reports and verify exit criteria.
- Analyze testing completion percentage and project quality before final release.

2.1 Items to Be Tested

- Login and Registration
- Google Login
- Appointment Booking
- Payment Gateway
- Zoom Meeting Integration
- Prescription Management
- Diet Plan Management
- Report Upload/Download
- Chat Functionality
- Healthcare Tools
- Appointment Sorting and Filtering
- Admin and Doctor Dashboard Features

2.2 Strategy of Testing

- Functional testing is performed to validate all healthcare workflows.
- Regression testing is performed after bug fixing to ensure existing functionality works properly.
- Automation testing is used for repetitive and critical modules like login, appointment booking, and reports.

2.3 Features to Be Tested

- User authentication and authorization
- Appointment management
- Online consultation workflow
- Report management
- Prescription and diet plan visibility
- Payment integration
- Chat and communication system
- Zoom meeting minimization and PiP mode

2.4 Test Case Matrix

Module	Planned Test Cases	Executed Test Cases
Login & Registration	20	20
Appointment Booking	25	25
Reports Management	15	15
Chat System	10	10
Zoom Meeting	18	18

2.5 Features Excluded from Testing

- Third-party internal API functionality
- Server-side infrastructure testing
- Payment gateway backend processing

- External Zoom server performance testing

2.6 Testing Approach

- Manual testing is performed for UI, usability, and workflow validation.
- Automation testing using Selenium is performed for repetitive and critical functionalities.
- Regression and smoke testing are executed after each build release.

2.7 Test Deliverables

- Test Plan Document
- Test Cases and Test Scenarios
- Bug Reports
- Requirement Traceability Matrix (RTM)
- Test Execution Report
- Test Summary Report

2.8 Bug Tracking System

- Defects are tracked using JIRA and Excel defect tracking sheets.
- Bugs are categorized based on severity, priority, status, and module impact.

2.9 Release Criteria

- All critical and high-severity defects must be fixed before release.
- Core functionalities such as login, booking, reports, payment, and meetings should work properly.

2.10 Test Case Pass/Fail Criteria

- A test case is marked Pass if the actual result matches the expected result.
- A test case is marked Fail if the actual result differs from the expected result.

2.11 Severity Codes

2.12 Severity Description

Critical	Application crash or major functionality failure
High	Important functionality not working
Medium	Partial functionality issue
Low	UI or minor issue

2.13 Suspension Criteria for Failed Smoke Test

- Testing is suspended if smoke testing fails for critical modules like login, payment, or meeting functionality.
- The build is returned to the development team for fixes before continuing testing.

2.14 Resumption Requirements

- All critical smoke test failures should be fixed successfully.
- Stable application build should be provided for retesting.

2.15 Release to User Acceptance Test (UAT) Criteria

- All critical and high-priority defects should be resolved.
- Major workflows should pass successfully before UAT release.

2.16 Exit Criteria

- All planned test cases should be executed successfully.
- Critical defects should be resolved and regression testing should pass.
- Test summary report and final documentation should be completed.

3 Implementation

3.1 Screenshots

- Screenshots are captured during testing for pass/fail validation and bug reporting.
- Screenshots are also used for documentation and requirement mapping.

3.2 Requirement Mapping

- Actual executed test cases are mapped with planned test cases to measure testing coverage.
- Requirement Traceability Matrix (RTM) is maintained for requirement tracking.

3.3 Percentage Completion of Testing

- Planned Testing Completion: 100%
- Executed Testing Completion: 95%
- Remaining testing includes pending bug fixes and retesting.

3.4 Major Tasks

- Requirement analysis
- Test case preparation
- Automation testing using Selenium
- Bug reporting and retesting
- Regression testing
- Report preparation and documentation

3.5 Security and Privacy

- User healthcare data is stored securely.
- Access control and authentication mechanisms are implemented to protect patient information.
- Payment and healthcare records are handled securely.

3.6 Training Requirements

- Users and healthcare professionals require training for appointment booking, online consultation, and report management.
- Admin and testing teams require training for system monitoring and defect management.

4. Testing

4.1 Alpha and Beta Testing

- Alpha testing is performed internally by the testing team before release.
- Beta testing is performed by selected end users to validate real-world usability and performance.

4.2 User Acceptance of Project

- Users and healthcare professionals validate the system functionality based on business requirements.
- User feedback is collected to improve usability and performance.

5. Limitations & Enhancements

5.1 Limitations

- Dependency on internet connectivity and third-party APIs.
- Zoom and payment gateway limitations may affect functionality.
- Some performance issues may occur during heavy traffic.

5.2 Enhancements

- Improve report loading performance.
- Add advanced healthcare analytics and AI-based recommendations.
- Improve PiP mode and meeting restoration functionality.
- Enhance chat system and notification management.